

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación y Diseño Digital · Ingeniería de Software

Seguridad en APIs REST con Spring Security y JWT

*Continuación del proyecto **appweb-api**: autenticación, roles y autorización en una API REST sin estado*

Spring Boot 4 · Spring Security 7 · JWT 0.13 · PostgreSQL · BCrypt

A partir del CRUD REST de la Semana 6: añadimos inicio de sesión, emisión y validación de tokens, y control de acceso por rol

Asignatura: Aplicaciones Web (5.º semestre)

Unidad: II — Lado del servidor y seguridad

Docente: Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.

Período: PPA 2026–2027

Versiones de referencia verificadas a junio de 2026 (Spring Boot 4.1.0, Spring Security 7.1, JWT 0.13.0, Java 25 LTS, PostgreSQL 18) en la sección 2.

Contenido

1	Introducción	3
1.1	Autenticación y autorización: dos preguntas distintas	3
1.2	Por qué JWT en una API REST	3
2	Versiones de referencia	3
3	Cómo funciona un JWT	4
4	Paso 1 — Dependencias y configuración	5
4.1	El archivo application.properties	6
5	Paso 2 — Usuario, roles y acceso a datos	7
5.1	Las tablas las crea Spring Boot, no tú	8
6	Paso 3 — UserDetailsService	9
7	Paso 4 — Codificador y AuthenticationManager	10
8	Paso 5 — Servicio de tokens (JwtService)	10
9	Paso 6 — Filtro de autenticación JWT	12
10	Paso 7 — Configuración de seguridad	14
11	Paso 8 — Controlador de autenticación	16
12	Paso 9 — Autorización por método	19
13	Paso 10 — Probar la seguridad	19
14	Paso 11 — Documentar la seguridad en Swagger	20
15	Clientes en otro servidor: CORS entre orígenes	21
15.1	Qué es un «origen» y la política del mismo origen	21
15.2	CORS solo afecta al navegador	22
15.3	La petición preflight	22
15.4	Configurar CORS en Spring Security	23

15.5 El lado del cliente Angular	24
15.6 El lado Thymeleaf de servidor a servidor	24
16 Buenas prácticas y errores comunes	25
17 Lista de verificación	25
18 Referencias	26

1 Introducción

Esta guía retoma exactamente el proyecto que construiste en la Semana 6 —la API REST `appweb-api` con Spring Boot, las entidades `Carrera`, `Estudiante`, `Perfil`, `Curso` e `Inscripcion`, y los controladores con los cinco verbos HTTP— y le añade una capa de seguridad profesional. Hasta ahora cualquier cliente podía consumir cualquier endpoint; al terminar, la API exigirá un inicio de sesión, emitirá un token firmado y autorizará cada petición según el rol del usuario.

El enfoque es el mismo de toda la asignatura: primero planteamos el problema concreto que vamos a resolver, después explicamos la técnica que lo resuelve y mostramos un ejemplo pequeño pero completo, y al final reunimos todas las piezas sobre el proyecto real. No se requiere ninguna librería ajena al ecosistema de Spring salvo una, `JWT`, para construir y verificar los tokens.

Qué vas a añadir al proyecto

Una entidad `Usuario` con sus roles, un `UserDetailsService` que la lee desde PostgreSQL, un codificador `BCrypt`, un servicio que emite y valida tokens JWT, un filtro que intercepta cada petición, una clase de configuración `SecurityConfig` y un `AuthController` con los endpoints de registro e inicio de sesión. Nada de lo ya construido se rompe: solo se protege.

1.1 Autenticación y autorización: dos preguntas distintas

Conviene separar dos conceptos que suelen confundirse. La **autenticación** responde a «¿quién eres?»: verifica la identidad del usuario comprobando sus credenciales (usuario y contraseña). La **autorización** responde a «¿qué puedes hacer?»: una vez identificado, decide si tiene permiso para ejecutar una acción concreta sobre un recurso. En esta guía resolvemos ambas: la autenticación al iniciar sesión, y la autorización en cada petición posterior según el rol.

1.2 Por qué JWT en una API REST

Una API REST es **sin estado** (stateless): el servidor no guarda una sesión en memoria entre una petición y la siguiente. Esto choca con el inicio de sesión tradicional basado en cookies de sesión, que sí guarda estado en el servidor. La solución estándar es el **token**: tras autenticarse una sola vez, el cliente recibe una credencial autocontenida que envía en cada petición; el servidor la valida sin necesidad de recordar nada. JWT (JSON Web Token) es el formato más extendido para ese token, y encaja de forma natural con una SPA en Angular que más adelante consumirá esta API.

2 Versiones de referencia

La seguridad es el área de Spring donde más cambian las APIs entre versiones. ¿Qué versiones exactas usamos y qué cambió respecto a tutoriales antiguos?

El proyecto se construyó sobre Spring Boot 4.1, que arrastra Spring Framework 7, Jakarta EE 11 (paquetes `jakarta.*`) y, lo más importante para nosotros, Spring Security 7. Esta generación

elimina APIs que llevaban años en desuso, por lo que mucho código que encontrarás en Internet ya no compila. La Tabla 1 fija las versiones verificadas; los cambios de API se detallan en la sección 5.

Tabla 1: Versiones de referencia verificadas a junio de 2026.

Componente	Versión	Rol en la seguridad
Spring Boot	4.1.0	Orquesta el arranque y la auto-configuración
Spring Security	7.1	Filtros, autenticación y autorización
Spring Framework	7.0.7+	Núcleo (DI, web MVC)
JJWT (io.jsonwebtoken)	0.13.0	Construye y verifica los tokens
Java	25 LTS (o 21 LTS)	Lenguaje y máquina virtual
PostgreSQL	18	Almacena usuarios y roles
Jackson	3 (en Spring)	Serializa JSON de la API

Spring Security 7 obliga al DSL de lambdas

En Spring Security 7 se eliminaron `authorizeRequests()`, el método `and()` para encadenar, y los emparejadores `antMatchers()` / `mvcMatchers()`. Ahora se usa `authorizeHttpRequests(...)` con lambdas y `requestMatchers(...)` (basado en `PathPattern` por defecto). Si copias código de un tutorial de Spring Security 5 o 6 con `.and()`, no compilará.

3 Cómo funciona un JWT

El cliente debe demostrar su identidad en cada petición sin reenviar la contraseña y sin que el servidor guarde sesión. ¿Cómo es esa credencial por dentro?

Un JWT es una cadena de texto compuesta por tres partes separadas por puntos: **cabecera**. **cuerpo**. **firma**. La **cabecera** (header) declara el algoritmo de firma; el **cuerpo** (payload) contiene las afirmaciones o «claims» (por ejemplo el nombre de usuario, los roles y la fecha de expiración); la **firma** se calcula con una clave secreta que solo conoce el servidor. Las dos primeras partes van codificadas en Base64URL —no cifradas—, así que nunca deben llevar datos sensibles; lo que protege al token es la firma: si alguien altera el cuerpo, la firma deja de coincidir y el token se rechaza.

El flujo completo tiene tres momentos. Primero el cliente envía usuario y contraseña al endpoint de inicio de sesión; si son correctos, el servidor responde con un token firmado. Después, en cada petición protegida, el cliente añade la cabecera **Authorization: Bearer <token>**. Finalmente el servidor verifica la firma y la expiración del token y, si todo está bien, atiende la petición con la identidad y los roles que vienen dentro. La Figura conceptual de la sección de pruebas muestra este intercambio con **cURL**.

Token firmado — no cifrado

Firmar garantiza *integridad* y *autenticidad* (nadie lo alteró y lo emitió nuestro servidor), pero no *confidencialidad*: cualquiera puede leer el cuerpo. Por eso el token viaja siempre sobre HTTPS y no contiene contraseñas ni datos privados, solo el identificador del usuario y sus roles.

4 Paso 1 — Dependencias y configuración

El proyecto actual no tiene capacidad de seguridad ni de manejo de tokens. ¿Qué dependencias añadimos al `pom.xml`?

Añadimos dos cosas al `pom.xml` de la Semana 6: el *starter* de seguridad de Spring Boot y las tres dependencias de JWT (la API, la implementación en tiempo de ejecución y el módulo de serialización). En cuanto añadas `spring-boot-starter-security`, Spring protegerá *todos* los endpoints por defecto; los iremos abriendo selectivamente en la sección 9.

pom.xml — dependencias nuevas

```

1 <!-- Spring Security -->
2 <dependency>
3   <groupId>org.springframework.boot</groupId>
4   <artifactId>spring-boot-starter-security</artifactId>
5 </dependency>
6
7 <!-- JSON Web Token (JWT 0.13.0) -->
8 <dependency>
9   <groupId>io.jsonwebtoken</groupId>
10  <artifactId>jjwt-api</artifactId>
11  <version>0.13.0</version>
12 </dependency>
13 <dependency>
14   <groupId>io.jsonwebtoken</groupId>
15   <artifactId>jjwt-impl</artifactId>
16   <version>0.13.0</version>
17   <scope>runtime</scope>
18 </dependency>
19 <dependency>
20   <groupId>io.jsonwebtoken</groupId>
21   <artifactId>jjwt-jackson</artifactId>
22   <version>0.13.0</version>
23   <scope>runtime</scope>
24 </dependency>

```

Convivencia de Jackson 2 y Jackson 3

Spring Boot 4 usa Jackson 3 (paquete `tools.jackson`). El módulo `jjwt-jackson` usa Jackson 2 (paquete `com.fasterxml.jackson`) solo de forma interna. Conviven sin conflicto porque sus paquetes raíz son distintos; JJWT trae su propia versión de Jackson 2 y no interfiere con la serialización de tu API. Si prefieres no tener Jackson 2 en el classpath, sustituye `jjwt-jackson` por `jjwt-gson`.

4.1 El archivo `application.properties`

Toda la configuración del proyecto vive en un único archivo, `src/main/resources/application.properties`: la conexión a la base de datos, el comportamiento de JPA/Hibernate y, ahora, las claves del JWT. No hay un archivo aparte para la base de datos ni otro para la seguridad; Spring Boot lee este archivo al arrancar. Como alternativa equivalente puede usarse `application.yml` (las mismas claves en formato YAML, anidadas por puntos); en esta guía usamos `.properties`.

El listado siguiente es el archivo completo del proyecto: incluye lo que ya tenías de la Semana 6 (origen de datos y JPA) más las dos claves nuevas del JWT al final. Para el algoritmo HS256 el secreto debe medir al menos 32 bytes; usa una cadena larga y aleatoria, y en producción léela de una variable de entorno, nunca del repositorio.

`src/main/resources/application.properties` — archivo completo

```

1 # --- Conexion a PostgreSQL ---
2 spring.datasource.url=jdbc:postgresql://localhost:5432/appweb2026ppa
3 spring.datasource.username=postgres
4 spring.datasource.password=P@$7Gr3$QL
5 spring.datasource.driver-class-name=org.postgresql.Driver
6
7 # --- JPA / Hibernate ---
8 spring.jpa.hibernate.ddl-auto=update
9 spring.jpa.show-sql=true
10 spring.jpa.properties.hibernate.format_sql=true
11
12 # --- Swagger / OpenAPI ---
13 springdoc.swagger-ui.path=/swagger-ui.html
14
15 # --- Seguridad: JWT ---
16 app.jwt.secret=cambia-esta-clave-larga-y-aleatoria-de-32-bytes-minimo
17 app.jwt.expiration-ms=3600000

```

La clave `spring.jpa.hibernate.ddl-auto` es la que hace que no escribas SQL: Hibernate genera o actualiza las tablas a partir de tus clases `@Entity`, como detallamos en el Paso 2.

5 Paso 2 — Usuario, roles y acceso a datos

Spring Security necesita saber qué usuarios existen, qué contraseña (cifrada) tienen y qué pueden hacer. ¿Cómo modelamos eso en el dominio del proyecto?

Creamos una entidad **Usuario** en el paquete **model** y un enumerado **Rol**. La contraseña se guarda *siempre* cifrada con BCrypt, nunca en claro. Los roles se modelan como una colección de valores del enumerado; Hibernate los almacena en una tabla secundaria **usuario_rol**. Mantenemos el estilo de la Semana 6: anotaciones JPA y Lombok para los accesorios.

model/Rol.java

```
1 package ec.edu.uteq.appwebapi.model;
2
3 public enum Rol {
4     ADMIN, DOCENTE, ESTUDIANTE
5 }
```

model/Usuario.java

```
1 package ec.edu.uteq.appwebapi.model;
2
3 import jakarta.persistence.*;
4 import lombok.Getter;
5 import lombok.Setter;
6 import java.util.Set;
7
8 @Entity
9 @Table(name = "usuario")
10 @Getter @Setter
11 public class Usuario {
12
13     @Id
14     @GeneratedValue(strategy = GenerationType.IDENTITY)
15     private Long id;
16
17     @Column(unique = true, nullable = false, length = 60)
18     private String username;
19
20     @Column(nullable = false)
21     private String password;           // hash BCrypt, nunca texto plano
22
23     @ElementCollection(fetch = FetchType.EAGER)
24     @CollectionTable(name = "usuario_rol",
25         joinColumns = @JoinColumn(name = "usuario_id"))
26     @Enumerated(EnumType.STRING)
27     @Column(name = "rol")
28     private Set<Rol> roles;
29 }
```

El repositorio es un **JpaRepository** con dos consultas derivadas: buscar por nombre de usuario (lo

usará el `UserDetailsService`) y comprobar si un usuario ya existe (lo usará el registro).

repository/UsuarioRepository.java

```

1 package ec.edu.uteq.appwebapi.repository;
2
3 import ec.edu.uteq.appwebapi.model.Usuario;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import java.util.Optional;
6
7 public interface UsuarioRepository extends JpaRepository<Usuario, Long> {
8     Optional<Usuario> findByUsername(String username);
9     boolean existsByUsername(String username);
10 }

```

5.1 Las tablas las crea Spring Boot, no tú

No escribimos sentencias `CREATE TABLE`. En el enfoque *code-first* de JPA, las clases `@Entity` son la única fuente de verdad del esquema: al arrancar, Hibernate las inspecciona y crea o actualiza automáticamente las tablas en PostgreSQL. La entidad `Usuario` produce la tabla `usuario`; su colección de roles, anotada con `@ElementCollection` y `@CollectionTable`, produce la tabla secundaria `usuario_rol` con su clave foránea. Si mañana añades un campo a la clase, basta con reiniciar la aplicación: la columna aparece sola.

Quien gobierna este comportamiento es la propiedad `spring.jpa.hibernate.ddl-auto` de `application.properties`. La Tabla 2 resume sus valores; para el desarrollo del proyecto usamos `update`.

Tabla 2: Valores de `spring.jpa.hibernate.ddl-auto`.

Valor	Qué hace
<code>update</code>	Crea las tablas que faltan y añade columnas nuevas; conserva los datos. Recomendado en desarrollo.
<code>create</code>	Borra y recrea el esquema en cada arranque; se pierden los datos. Útil para pruebas limpias.
<code>create-drop</code>	Como <code>create</code> , pero además borra el esquema al apagar la aplicación.
<code>validate</code>	No modifica nada; solo verifica que las tablas coincidan con las entidades y falla si no.
<code>none</code>	Desactiva la gestión automática; el esquema se administra por fuera (por ejemplo con Flyway).

Ver el SQL que genera Hibernate

Si quieres comprobar exactamente qué tablas crea, activa `spring.jpa.show-sql=true` y `spring.jpa.properties.hibernate.format_sql=true`: el SQL aparece formateado en la consola al arrancar. Es solo para inspección; no tienes que ejecutarlo a mano.

6 Paso 3 — UserDetailsService

Spring Security no sabe leer tu tabla `usuario`. ¿Cómo le enseñamos a cargar un usuario desde la base de datos?

El contrato `UserDetailsService` tiene un solo método, `loadUserByUsername`, que Spring invoca durante la autenticación. Nuestra implementación busca el `Usuario` en el repositorio, traduce sus roles a autoridades de Spring (con el prefijo `ROLE_`, que es la convención que esperan `hasRole(...)` y `@PreAuthorize`) y devuelve un `UserDetails` estándar.

`security/UsuarioDetailsService.java`

```

1 package ec.edu.uteq.appwebapi.security;
2
3 import ec.edu.uteq.appwebapi.model.Usuario;
4 import ec.edu.uteq.appwebapi.repository.UsuarioRepository;
5 import lombok.RequiredArgsConstructor;
6 import org.springframework.security.core.authority.SimpleGrantedAuthority;
7 import org.springframework.security.core.userdetails.*;
8 import org.springframework.stereotype.Service;
9
10 @Service
11 @RequiredArgsConstructor
12 public class UsuarioDetailsService implements UserDetailsService {
13
14     private final UsuarioRepository repo;
15
16     @Override
17     public UserDetails loadUserByUsername(String username) {
18         Usuario u = repo.findByUsername(username)
19             .orElseThrow(() ->
20                 new UsernameNotFoundException("Usuario no encontrado: " + username));
21
22         var autoridades = u.getRoles().stream()
23             .map(r -> new SimpleGrantedAuthority("ROLE_" + r.name()))
24             .toList();
25
26         return User.builder()
27             .username(u.getUsername())
28             .password(u.getPassword()) // hash BCrypt ya almacenado
29             .authorities(autoridades)
30             .build();
31     }
32 }
```

El prefijo `ROLE_` es una convención

`hasRole("ADMIN")` comprueba internamente la autoridad `ROLE_ADMIN`. Por eso al construir las autoridades antepone `ROLE_` al nombre del enumerado. Si en cambio usaras `hasAuthority(...)`, deberías escribir el nombre completo de la autoridad sin que Spring añada el prefijo.

7 Paso 4 — Codificador y AuthenticationManager

Las contraseñas no pueden guardarse en claro, y alguien debe orquestar la comprobación de credenciales. ¿Quién cifra y quién autentica?

Necesitamos dos beans. El `PasswordEncoder` (usamos `BCryptPasswordEncoder`) cifra las contraseñas al registrar y las compara al iniciar sesión. El `AuthenticationManager` es quien realmente autentica; en Spring Security 7 lo construimos a partir de un `DaoAuthenticationProvider` —que recibe nuestro `UserDetailsService` por constructor y el codificador por *setter*— envuelto en un `ProviderManager`. Este es el patrón oficial de la documentación de Spring Security 7.

Beans de seguridad (dentro de SecurityConfig)

```

1  @Bean
2  public PasswordEncoder passwordEncoder() {
3      return new BCryptPasswordEncoder();
4  }
5
6  @Bean
7  public AuthenticationManager authenticationManager(
8      UserDetailsService userDetailsService,
9      PasswordEncoder passwordEncoder) {
10     DaoAuthenticationProvider provider =
11         new DaoAuthenticationProvider(userDetailsService);
12     provider.setPasswordEncoder(passwordEncoder);
13     return new ProviderManager(provider);
14 }
```

Constructor nuevo de DaoAuthenticationProvider

En Spring Security 7 el constructor sin argumentos con `setUserDetailsService(...)` está en desuso. La forma correcta y actual es `new DaoAuthenticationProvider(userDetailsService)` y, a continuación, `setPasswordEncoder(...)`. Mezclar el estilo antiguo con el nuevo es la causa más común del error «constructor cannot be applied».

8 Paso 5 — Servicio de tokens (JwtService)

Tras autenticar a un usuario hay que emitirle un token, y en cada petición hay que leerlo y validarlo. ¿Dónde concentramos esa lógica?

Aislamos todo el manejo de JWT en un `JwtService`. Construye la clave de firma a partir del secreto, genera un token con el nombre de usuario como *subject*, los roles como un *claim* y una expiración; y, en sentido inverso, extrae el usuario de un token y comprueba su validez. Usamos la

API moderna de JJWT 0.13: `Jwts.builder()` para emitir y `Jwts.parser().verifyWith(...).build().parseSignedClaims(...)` para verificar.

security/JwtService.java — parte 1: emisión

```
1 package ec.edu.uteq.appwebapi.security;
2
3 import io.jsonwebtoken.Claims;
4 import io.jsonwebtoken.Jwts;
5 import io.jsonwebtoken.security.Keys;
6 import org.springframework.beans.factory.annotation.Value;
7 import org.springframework.security.core.GrantedAuthority;
8 import org.springframework.security.core.userdetails.UserDetails;
9 import org.springframework.stereotype.Service;
10
11 import javax.crypto.SecretKey;
12 import java.nio.charset.StandardCharsets;
13 import java.time.Instant;
14 import java.util.Date;
15
16 @Service
17 public class JwtService {
18
19     @Value("${app.jwt.secret}")
20     private String secret;
21
22     @Value("${app.jwt.expiration-ms}")
23     private long expirationMs;
24
25     private SecretKey clave() {
26         return Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
27     }
28
29     public String generarToken(UserDetails usuario) {
30         Instant ahora = Instant.now();
31         return Jwts.builder()
32             .subject(usuario.getUsername())
33             .claim("roles", usuario.getAuthorities().stream()
34                 .map(GrantedAuthority::getAuthority).toList())
35             .issuedAt(Date.from(ahora))
36             .expiration(Date.from(ahora.plusMillis(expirationMs)))
37             .signWith(clave())
38             .compact();
39     }
40 }
```

security/JwtService.java — parte 2: lectura y validación

```

1  public String extraerUsername(String token) {
2      return parsear(token).getSubject();
3  }
4
5  public boolean esValido(String token, UserDetails usuario) {
6      Claims c = parsear(token);
7      return c.getSubject().equals(usuario.getUsername())
8          && c.getExpiration().after(new Date());
9  }
10
11 private Claims parsear(String token) {
12     return Jwts.parser()
13         .verifyWith(clave())
14         .build()
15         .parseSignedClaims(token)
16         .getPayload();
17 }
18 }

```

El algoritmo lo elige la clave

Con `signWith(clave())` no indicamos el algoritmo a mano: JJWT lo deduce del tamaño de la clave HMAC. Una clave de 32 bytes produce HS256; una mayor, HS384 o HS512. Por eso basta con que el secreto cumpla el tamaño mínimo.

9 Paso 6 — Filtro de autenticación JWT

Cada petición protegida llega con su token en la cabecera, pero Spring aún no lo mira. ¿Cómo interceptamos la petición para validar el token antes de que llegue al controlador?

Extendemos `OncePerRequestFilter`, que garantiza una sola ejecución por petición. El filtro lee la cabecera `Authorization`, y si trae un `Bearer`, extrae el token, recupera el usuario, valida y —si todo está bien— coloca la autenticación en el `SecurityContext`. A partir de ese momento Spring considera autenticada la petición y aplica las reglas de autorización. Si el token falta o es inválido, el filtro simplemente deja pasar la petición sin autenticar, y la cadena de seguridad responderá 401 o 403 según corresponda.

security/JwtAuthFilter.java — parte 1: imports y campos

```
1 package ec.edu.uteq.appwebapi.security;
2
3 import io.jsonwebtoken.JwtException;
4 import jakarta.servlet.FilterChain;
5 import jakarta.servlet.ServletException;
6 import jakarta.servlet.http.HttpServletRequest;
7 import jakarta.servlet.http.HttpServletResponse;
8 import lombok.RequiredArgsConstructor;
9 import org.springframework.lang.NonNull;
10 import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
11 import org.springframework.security.core.context.SecurityContextHolder;
12 import org.springframework.security.core.userdetails.UserDetails;
13 import org.springframework.security.core.userdetails.UserDetailsService;
14 import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
15 import org.springframework.stereotype.Component;
16 import org.springframework.web.filter.OncePerRequestFilter;
17
18 import java.io.IOException;
19
20 @Component
21 @RequiredArgsConstructor
22 public class JwtAuthFilter extends OncePerRequestFilter {
23
24     private final JwtService jwtService;
25     private final UserDetailsService userDetailsService;
```

security/JwtAuthFilter.java — parte 2: doFilterInternal

```

1  @Override
2  protected void doFilterInternal(@NonNull HttpServletRequest req,
3      @NonNull HttpServletResponse res, @NonNull FilterChain chain)
4      throws ServletException, IOException {
5
6      String cabecera = req.getHeader("Authorization");
7      if (cabecera == null || !cabecera.startsWith("Bearer ")) {
8          chain.doFilter(req, res);
9          return;
10     }
11     String token = cabecera.substring(7);
12     try {
13         String username = jwtService.extraerUsername(token);
14         if (username != null
15             && SecurityContextHolder.getContext().getAuthentication() == null) {
16             UserDetails usuario = userDetailsService.loadUserByUsername(username);
17             if (jwtService.esValido(token, usuario)) {
18                 var auth = new UsernamePasswordAuthenticationToken(
19                     usuario, null, usuario.getAuthorities());
20                 auth.setDetails(
21                     new WebAuthenticationDetailsSource().buildDetails(req));
22                 SecurityContextHolder.getContext().setAuthentication(auth);
23             }
24         }
25     } catch (JwtException e) {
26         // token invalido o expirado: se continua sin autenticar
27     }
28     chain.doFilter(req, res);
29 }
30 }

```

10 Paso 7 — Configuración de seguridad

Falta unir todo: declarar qué rutas son públicas, cuáles exigen rol, desactivar el estado de sesión y enchufar nuestro filtro. ¿Dónde se define la política?

La clase `SecurityConfig` expone el `SecurityFilterChain`. Desactivamos CSRF (es seguro hacerlo *solo* porque la API es sin estado y no usa cookies de sesión), declaramos públicos los endpoints de autenticación y de Swagger, exigimos rol en el resto, fijamos la política de sesión en `STATELESS` y registramos nuestro `JwtAuthFilter` antes del filtro de usuario y contraseña. La anotación `@EnableMethodSecurity` habilita además `@PreAuthorize` en los controladores.

security/SecurityConfig.java — parte 1

```
1 package ec.edu.uteq.appwebapi.security;
2
3 import lombok.RequiredArgsConstructor;
4 import org.springframework.context.annotation.*;
5 import org.springframework.http.HttpMethod;
6 import org.springframework.security.authentication.*;
7 import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
8 import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
9 import org.springframework.security.config.annotation.web.builders.HttpSecurity;
10 import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
11 import org.springframework.security.config.http.SessionCreationPolicy;
12 import org.springframework.security.core.userdetails.UserDetailsService;
13 import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
14 import org.springframework.security.crypto.password.PasswordEncoder;
15 import org.springframework.security.web.SecurityFilterChain;
16 import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
17
18 @Configuration
19 @EnableWebSecurity
20 @EnableMethodSecurity
21 @RequiredArgsConstructor
22 public class SecurityConfig {
23
24     private final JwtAuthFilter jwtAuthFilter;
25     private final UserDetailsService userDetailsService;
```


security/SecurityConfig.java — parte 2: la cadena de filtros

```

1  @Bean
2  public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
3      http
4          .csrf(csrf -> csrf.disable())
5          .authorizeHttpRequests(auth -> auth
6              .requestMatchers("/api/auth/**").permitAll()
7              .requestMatchers("/swagger-ui/**", "/v3/api-docs/**").permitAll()
8              .requestMatchers(HttpMethod.GET, "/api/estudiantes/**")
9                  .hasAnyRole("ADMIN", "DOCENTE", "ESTUDIANTE")
10             .requestMatchers(HttpMethod.POST, "/api/estudiantes/**")
11                 .hasAnyRole("ADMIN", "DOCENTE")
12             .requestMatchers(HttpMethod.DELETE, "/api/**").hasRole("ADMIN")
13             .anyRequest().authenticated())
14          .sessionManagement(sm ->
15              sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
16          .addFilterBefore(jwtAuthFilter,
17              UsernamePasswordAuthenticationFilter.class);
18      return http.build();
19  }
20
21  @Bean
22  public PasswordEncoder passwordEncoder() {
23      return new BCryptPasswordEncoder();
24  }
25
26  @Bean
27  public AuthenticationManager authenticationManager() {
28      DaoAuthenticationProvider provider =
29          new DaoAuthenticationProvider(userDetailsService);
30      provider.setPasswordEncoder(passwordEncoder());
31      return new ProviderManager(provider);
32  }
33  }

```

No desactives CSRF en aplicaciones con sesión

Aquí `csrf.disable()` es correcto porque la API es sin estado y se autentica con un token en la cabecera, no con cookies. En una aplicación web tradicional con sesión y formularios, desactivar CSRF abre la puerta a ataques de falsificación de petición. La regla es: token en cabecera → CSRF no aplica; cookie de sesión → CSRF obligatorio.

11 Paso 8 — Controlador de autenticación

Hacen falta los puntos de entrada que el cliente usará para registrarse y para iniciar sesión y recibir su token. ¿Cómo son esos endpoints?

El `AuthController` expone `/api/auth/registro` y `/api/auth/login`. El registro cifra la contraseña y guarda el usuario; el login delega en el `AuthenticationManager` —creando un token de

autenticación *no autenticado* con `unauthenticated(...)`, como indica la documentación oficial— y, si las credenciales son válidas, emite el JWT. Los datos de entrada y salida viajan en `record` de Java, igual que los DTO de la Semana 6.

dto/AuthDtos.java — contratos de entrada y salida

```
1 package ec.edu.uteq.appwebapi.dto;
2
3 import ec.edu.uteq.appwebapi.model.Rol;
4 import java.util.Set;
5
6 public record LoginRequest(String username, String password) {}
7 public record RegistroRequest(String username, String password, Set<Rol> roles) {}
8 public record AuthResponse(String token, String tipo, long expiraEnMs) {}
```

controller/AuthController.java

```

1 package ec.edu.uteq.appwebapi.controller;
2
3 import ec.edu.uteq.appwebapi.dto.*;
4 import ec.edu.uteq.appwebapi.model.Usuario;
5 import ec.edu.uteq.appwebapi.repository.UsuarioRepository;
6 import ec.edu.uteq.appwebapi.security.JwtService;
7 import lombok.RequiredArgsConstructor;
8 import org.springframework.http.*;
9 import org.springframework.security.authentication.*;
10 import org.springframework.security.core.Authentication;
11 import org.springframework.security.core.userdetails.UserDetails;
12 import org.springframework.security.crypto.password.PasswordEncoder;
13 import org.springframework.web.bind.annotation.*;
14
15 @RestController
16 @RequestMapping("/api/auth")
17 @RequiredArgsConstructor
18 public class AuthController {
19
20     private final AuthenticationManager authenticationManager;
21     private final JwtService jwtService;
22     private final UsuarioRepository usuarioRepository;
23     private final PasswordEncoder passwordEncoder;
24
25     @PostMapping("/registro")
26     public ResponseEntity<String> registrar(@RequestBody RegistroRequest req) {
27         if (usuarioRepository.existsByUsername(req.username())) {
28             return ResponseEntity.badRequest().body("El usuario ya existe");
29         }
30         Usuario u = new Usuario();
31         u.setUsername(req.username());
32         u.setPassword(passwordEncoder.encode(req.password()));
33         u.setRoles(req.roles());
34         usuarioRepository.save(u);
35         return ResponseEntity.status(HttpStatus.CREATED).body("Usuario registrado");
36     }
37
38     @PostMapping("/login")
39     public ResponseEntity<AuthResponse> login(@RequestBody LoginRequest req) {
40         Authentication petition = UsernamePasswordAuthenticationToken
41             .unauthenticated(req.username(), req.password());
42         Authentication auth = authenticationManager.authenticate(petition);
43         String token = jwtService.generarToken((UserDetails) auth.getPrincipal());
44         return ResponseEntity.ok(new AuthResponse(token, "Bearer", 3600000));
45     }
46 }

```

12 Paso 9 — Autorización por método

Las reglas por ruta de `SecurityConfig` cubren lo general, pero a veces queremos proteger un método concreto de un controlador existente. ¿Cómo se hace sin tocar la configuración global?

Gracias a `@EnableMethodSecurity`, podemos anotar métodos con `@PreAuthorize` y una expresión de seguridad. Es la forma más legible de blindar acciones sensibles: por ejemplo, restringir el borrado de un estudiante únicamente al rol `ADMIN`, directamente sobre el `EstudianteController` de la Semana 6.

controller/EstudianteController.java — método protegido

```
1 @DeleteMapping("/{id}")
2 @PreAuthorize("hasRole('ADMIN')")
3 @ResponseStatus(HttpStatus.NO_CONTENT)
4 public void eliminar(@PathVariable Long id) {
5     servicio.eliminar(id);
6 }
```

Dos capas que se complementan

La autorización por ruta (en `SecurityConfig`) y la autorización por método (`@PreAuthorize`) no se excluyen: la primera filtra pronto, antes de entrar al controlador; la segunda protege puntos concretos con reglas finas. Usar ambas es una defensa en profundidad.

13 Paso 10 — Probar la seguridad

Con todo en su sitio, hay que comprobar el ciclo completo: registrar, iniciar sesión, recibir el token y usarlo. ¿Cómo lo verificamos sin un frontend?

Usamos `cURL` para recorrer el flujo. Primero registramos un administrador; luego iniciamos sesión y copiamos el token de la respuesta; por último llamamos a un endpoint protegido enviando ese token en la cabecera `Authorization`. Una llamada sin token a un recurso protegido debe devolver 401; con un rol insuficiente, 403.

Terminal — ciclo completo de autenticación

```
1 # 1) Registrar un administrador
2 curl -X POST http://localhost:8080/api/auth/registro \
3     -H "Content-Type: application/json" \
4     -d '{"username":"admin","password":"Admin123*","roles":["ADMIN"]}'
5
6 # 2) Iniciar sesion y obtener el token
7 curl -X POST http://localhost:8080/api/auth/login \
8     -H "Content-Type: application/json" \
9     -d '{"username":"admin","password":"Admin123*"}'
10 # Respuesta: {"token":"eyJhbGciOiJIUzI1NiJ9...", "tipo":"Bearer", "expiraEnMs":3600000}
11
12 # 3) Llamar a un endpoint protegido con el token
13 curl http://localhost:8080/api/estudiantes \
14     -H "Authorization: Bearer eyJhbGciOiJIUzI1NiJ9..."
15
16 # 4) Llamar sin token -> 401 Unauthorized
17 curl -i http://localhost:8080/api/estudiantes
```

14 Paso 11 — Documentar la seguridad en Swagger

La Semana 6 dejó Swagger UI funcionando, pero ahora los endpoints exigen token. ¿Cómo hacemos que Swagger permita escribir el **Bearer** para probar?

Con `springdoc-openapi` declaramos un esquema de seguridad de tipo HTTP **bearer** mediante anotaciones. Así aparece el botón «Authorize» en Swagger UI, donde el usuario pega su token y prueba los endpoints protegidos desde el navegador. Recuerda que en `SecurityConfig` ya dejamos públicas las rutas `/swagger-ui/**` y `/v3/api-docs/**`.

config/OpenApiConfig.java

```

1 package ec.edu.uteq.appwebapi.config;
2
3 import io.swagger.v3.oas.annotations.OpenAPIDefinition;
4 import io.swagger.v3.oas.annotations.enums.SecuritySchemeType;
5 import io.swagger.v3.oas.annotations.info.Info;
6 import io.swagger.v3.oas.annotations.security.SecurityRequirement;
7 import io.swagger.v3.oas.annotations.security.SecurityScheme;
8 import org.springframework.context.annotation.Configuration;
9
10 @Configuration
11 @OpenAPIDefinition(
12     info = @Info(title = "API Academica UTEQ", version = "1.0"),
13     security = @SecurityRequirement(name = "bearerAuth"))
14 @SecurityScheme(
15     name = "bearerAuth",
16     type = SecuritySchemeType.HTTP,
17     scheme = "bearer",
18     bearerFormat = "JWT")
19 public class OpenApiConfig {
20 }

```

15 Clientes en otro servidor: CORS entre orígenes

El cliente que consume la API (una SPA en Angular, o una página con JavaScript) se ejecuta en el navegador desde un origen distinto al de la API. Por seguridad, el navegador bloquea esas llamadas. ¿Cómo autorizamos al cliente legítimo sin abrir la API a cualquiera?

Hasta aquí la guía asumió que el cliente y la API comparten origen o que el cliente no es un navegador. En un despliegue real es habitual separar los servidores: la API REST en `api.uteq.edu.ec` y el cliente en `app.uteq.edu.ec` o, en desarrollo, Angular en `localhost:4200` y la API en `localhost:8080`. Esta sección explica cuándo eso exige configurar CORS y cómo hacerlo en Spring Security 7.

15.1 Qué es un «origen» y la política del mismo origen

Un **origen** es la combinación de esquema, host y puerto. Así, `https://app.uteq.edu.ec` y `https://api.uteq.edu.ec` son orígenes distintos (host diferente), y `http://localhost:4200` y `http://localhost:8080` también lo son (puerto diferente). Por la **política del mismo origen**, el navegador impide que el JavaScript de un origen lea la respuesta de otro, salvo que el servidor lo permita explícitamente. CORS (Cross-Origin Resource Sharing) es justamente el mecanismo por el que el servidor declara qué orígenes externos pueden consumir sus recursos.

15.2 CORS solo afecta al navegador

Este es el punto que más confusión genera, y la razón por la que Thymeleaf y Angular no se comportan igual. CORS es una política que **aplica el navegador**; solo interviene cuando el código que hace la petición se ejecuta dentro de un navegador. La Tabla 3 resume los casos típicos del proyecto.

Tabla 3: Cuándo aplica CORS según el tipo de cliente.

Cliente	Dónde se ejecuta la llamada	¿Aplica CORS?
SPA Angular en otro servidor	Navegador	Sí
Thymeleaf (SSR) que llama por <code>RestClient</code>	Servidor (backend)	No
Thymeleaf con <code>fetch</code> /JS al API en otro origen	Navegador	Sí
Postman / <code>cURL</code> / pruebas	Herramienta (sin navegador)	No

En el caso de **Angular**, el navegador de la persona descarga la SPA desde el servidor del cliente y, desde ahí, llama a la API en otro origen: es una llamada entre orígenes y CORS aplica. En el caso de **Thymeleaf renderizado en el servidor**, el navegador solo habla con el servidor Thymeleaf (mismo origen para él), y es ese servidor quien llama a la API de backend a backend con `RestClient` o `WebClient`; esa llamada no pasa por el navegador, así que no está sujeta a CORS. El JWT viaja igualmente en la cabecera `Authorization` de esa llamada servidor-a-servidor, pero no hay preflight ni cabeceras CORS de por medio.

Postman y `cURL` no disparan CORS

Si pruebas la API con Postman o `cURL` y funciona, pero falla desde el navegador, no es un fallo de la API: es CORS. Esas herramientas no aplican la política del mismo origen; el navegador sí. Por eso conviene probar siempre el flujo real desde el cliente web.

15.3 La petición preflight

Para las peticiones que no son «simples» —y las nuestras no lo son, porque llevan la cabecera `Authorization`, usan métodos como `PUT` o `DELETE` y envían `Content-Type: application/json`— el navegador envía primero una petición `OPTIONS` llamada *preflight*. En ella pregunta al servidor si permite ese origen, ese método y esas cabeceras. El servidor responde con `Access-Control-Allow-Origin`, `-Methods` y `-Headers`; solo si la respuesta lo autoriza, el navegador envía la petición real. En la práctica, toda llamada protegida desde Angular dispara un preflight.

15.4 Configurar CORS en Spring Security

La regla de oro es habilitar CORS **dentro de Spring Security**, no solo en Spring MVC. La cadena de filtros de seguridad se ejecuta antes que el controlador; si el preflight `OPTIONS` es rechazado en esa capa, el navegador nunca recibe las cabeceras y la llamada falla aunque el controlador sea correcto. Por eso añadimos dos cosas a la configuración de la sección «Paso 7»: la línea `.cors(...)` en la cadena de filtros y un bean `corsConfigurationSource` con los orígenes permitidos.

SecurityConfig.java — línea a añadir en la cadena de filtros

```
1 http
2     .cors(Customizer.withDefaults())    // habilita CORS usando el bean de abajo
3     .csrf(csrf -> csrf.disable())
4     // ... resto de la configuracion de Paso 7 sin cambios ...
```

SecurityConfig.java — bean de configuración CORS

```
1 @Bean
2 public CorsConfigurationSource corsConfigurationSource() {
3     CorsConfiguration config = new CorsConfiguration();
4     config.setAllowedOrigins(List.of(
5         "https://app.uteq.edu.ec",    // SPA Angular en produccion
6         "http://localhost:4200"));   // Angular en desarrollo
7     config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
8     config.setAllowedHeaders(List.of("Authorization", "Content-Type"));
9     config.setExposedHeaders(List.of("Authorization"));
10    config.setAllowCredentials(false);    // token en cabecera, no cookies
11    config.setMaxAge(Duration.ofHours(1)); // cachea el preflight 1 hora
12
13    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
14    source.registerCorsConfiguration("/**", config);
15    return source;
16 }
```

Los imports que añade esta configuración son `org.springframework.web.cors.*`, `org.springframework.security.config.Customizer`, `java.util.List` y `java.time.Duration`.

El bean debe llamarse exactamente `corsConfigurationSource`

Con `.cors(Customizer.withDefaults())`, Spring Security busca un bean con el nombre exacto `corsConfigurationSource` y de tipo `UrlBasedCorsConfigurationSource`. Si lo nombras distinto, no lo detecta y el preflight se rechaza en silencio. En ese caso, pásalo explícito: `.cors(cors -> cors.configurationSource(miBean))`.

Nunca combines comodín de origen con credenciales

No uses `setAllowedOrigins(List.of("*"))` junto con `setAllowCredentials(true)`: el navegador rechaza esa combinación y, además, expondría la API a cualquier origen. Enumera los orígenes concretos de tu SPA. Si necesitas comodines controlados (por ejemplo varios subdominios), usa `setAllowedOriginPatterns(...)` en lugar de `setAllowedOrigins(...)`.

15.5 El lado del cliente Angular

En Angular, lo natural es un interceptor HTTP que añade la cabecera **Authorization: Bearer** a cada petición de forma automática, para no repetirla en cada servicio. El token se obtuvo del endpoint de inicio de sesión y se guarda en el cliente.

auth.interceptor.ts — Angular adjunta el token

```
1 export const authInterceptor: HttpInterceptorFn = (req, next) => {
2   const token = localStorage.getItem('token');
3   if (token) {
4     req = req.clone({
5       headers: { Authorization: `Bearer ${token}` }
6     });
7   }
8   return next(req);
9 };
```

Dónde guardar el token en el navegador

Guardar el token en `localStorage` es lo más común y sencillo, pero es legible por cualquier script, de modo que un ataque XSS podría robarlo. Para mayor seguridad puede usarse una cookie `HttpOnly`; eso, sin embargo, reintroduce la necesidad de CSRF y de `allowCredentials=true`. Es una decisión de diseño entre simplicidad y exposición.

15.6 El lado Thymeleaf de servidor a servidor

Cuando el cliente Thymeleaf llama a la API desde su backend, no hay CORS: el reto es otro, custodiar el token. El servidor Thymeleaf obtiene el JWT tras el inicio de sesión del usuario, lo guarda (por ejemplo en la sesión del usuario en ese servidor) y lo adjunta en cada llamada con `RestClient`, el cliente HTTP síncrono de Spring 6.1+.

Servidor Thymeleaf — consumo de la API con RestClient

```
1 // 'token' obtenido tras el login del usuario y guardado en su sesion
2 EstudianteDTO[] estudiantes = restClient.get()
3   .uri("https://api.uteq.edu.ec/api/estudiantes")
4   .header("Authorization", "Bearer " + token)
5   .retrieve()
6   .body(EstudianteDTO[].class);
```

Resumen de decisión

Si el cliente corre en el navegador y está en otro origen (Angular, o JavaScript en una página), configura CORS en Spring Security. Si el cliente es otro servidor que llama de backend a backend (Thymeleaf con `RestClient`), no necesitas CORS: solo enviar el token en la cabecera y proteger esa comunicación con HTTPS y, si procede, restricciones de red.

16 Buenas prácticas y errores comunes

La seguridad mal aplicada da una falsa sensación de protección. Estas recomendaciones evitan los fallos más frecuentes en proyectos de estudiantes y también en producción.

- **El secreto fuera del código.** Nunca subas `app.jwt.secret` al repositorio. Léelo de una variable de entorno y usa una clave larga y aleatoria; para HS256, mínimo 32 bytes.
- **Siempre HTTPS.** El token viaja en cada petición; sobre HTTP plano cualquiera puede interceptarlo y suplantar al usuario.
- **Expiración corta.** Una hora es un buen punto de partida. Para sesiones largas, lo correcto es añadir un *refresh token* en lugar de alargar el token de acceso.
- **No guardar datos sensibles en el cuerpo.** El payload es legible por cualquiera; incluye solo el identificador del usuario y sus roles.
- **Contraseñas con BCrypt.** Jamás se guardan en claro ni con hash simple (MD5/SHA). BCrypt incorpora sal y coste configurable.
- **Distinguir 401 de 403.** 401 (no autenticado) significa «no sé quién eres»; 403 (prohibido) significa «sé quién eres, pero no puedes». Confundirlos despista al cliente.

Error frecuente: orden y registro del filtro

Si el filtro no se ejecuta, revisa dos cosas: que esté anotado con `@Component` (para que Spring lo descubra) y que en `SecurityConfig` se registre con `addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class)`. Si lo registraras después, la cadena ya habría decidido rechazar la petición.

17 Lista de verificación

Antes de dar por terminada la integración de seguridad, confirma cada punto:

- Dependencias `spring-boot-starter-security` y `jjwt 0.13.0` en el `pom.xml`.
- Propiedades `app.jwt.secret` (≥ 32 bytes) y `app.jwt.expiration-ms` definidas.
- Entidad `Usuario`, enumerado `Rol` y `UsuarioRepository` creados; tablas creadas automáticamente por Hibernate desde las `@Entity` (`ddl-auto=update`).
- `UsuarioDetailsService` carga el usuario y mapea roles a `ROLE_*`.
- Beans `PasswordEncoder` (BCrypt) y `AuthenticationManager` (`DaoAuthenticationProvider + ProviderManager`).
- `JwtService` emite y valida tokens con la API de JJWT 0.13.
- `JwtAuthFilter` extiende `OncePerRequestFilter` y coloca la autenticación en el contexto.
- `SecurityConfig` con DSL de lambdas, sesión `STATELESS`, CSRF desactivado y filtro registrado.
- `AuthController` con `/registro` y `/login`; rutas públicas permitidas.

- Endpoints protegidos por rol (ruta y/o `@PreAuthorize`); Swagger con esquema `bearerAuth`.
- CORS habilitado en Spring Security (`.cors(...)` + bean `corsConfigurationSource`) cuando el cliente es un navegador en otro origen (Angular); con orígenes explícitos y sin comodín junto a credenciales.
- Ciclo probado con `cURL`: 201 al registrar, token al iniciar sesión, 200 con token, 401 sin token, 403 con rol insuficiente.

18 Referencias

Documentación oficial verificada (junio de 2026):

- Spring Security — Autenticación usuario/contraseña y `AuthenticationManager`:
<https://docs.spring.io/spring-security/reference/servlet/authentication/passwords/index.html>
- Spring Security — Integración de CORS:
<https://docs.spring.io/spring-security/reference/servlet/integrations/cors.html>
- Spring Security — `DaoAuthenticationProvider`:
<https://docs.spring.io/spring-security/reference/servlet/authentication/passwords/dao-authentication-provider.html>
- Spring Security 7 — Migración de configuración (DSL de lambdas):
<https://docs.spring.io/spring-security/reference/migration-7/configuration.html>
- JJWT (Java JWT) — repositorio y API 0.13:
<https://github.com/jwtkt/jjwt>
- Spring Boot — notas de versión 4.x:
<https://github.com/spring-projects/spring-boot/releases>
- springdoc-openapi — integración de OpenAPI 3 y Swagger UI:
<https://springdoc.org/>